Proposed Pull Requests — grblHAL Simulator

Fork `stevenrwood/Simulator` → upstream `grblHAL/Simulator` · base branch `master` (`171a661` "Updated submodules") · 5 proposal branches + 2 submodule forks · **none submitted upstream yet** (staging document)

How to consume this — it's a bundle.

- **Use the whole thing.** Clone `stevenrwood/Simulator` at `integration` with `--recurse-submodules` (it pins the `core` and `Plugin_SD_card` forks, so the firmware fixes ride along — no separate step), build with CMake + MinGW, and run it against **ioSender at its `integration` branch**. That pairing is what enables the 3D carve, ATC macro upload and littlefs macros.
- **Just a virtual controller?** Any of these branches, built on its own, gives a working simulator (homing, jogging, plain g-code, manual probing) for *any* grbl sender; only the headline features need the matching ioSender (see the dependency matrix below).
- **Upstreaming one piece.** Each card is a focused branch off `grblHAL/Simulator master`; the two submodule forks (S1 / S2) go to `grblHAL/core` and `grblHAL/Plugin_SD_card`. The granular cards exist for that — for *using* it, take the two integrations.

Shared base. All five branches start from the same 6-commit *sim-core* base (below): simulated limit/probe switches, fast homing, and the CI release workflow. `pr/sim-view` (= the fork's `integration` branch, `9f7fdab`) is a **superset** that contains every other branch plus the 3D view; the focused branches are its constituent, separately-reviewable parts. Line counts below **exclude vendored code** (the littlefs library and the `src/grbl` / `src/sdcard` submodules — the latter are tracked as the two submodule-fork PRs at the end).

Features at a glance — **NEW** new capability, **CHG** changes existing behaviour, **FIX** bug fix. Each is a focused branch off `master`; `pr/sim-view` (PR 5) is the integration superset of the rest. The two submodule-fork fixes (S1, S2) ride with PRs 3 and 1.

Simulator capabilities

Simulated limit/probe switches, fast homing, and a CI release workflow (the shared base of every branch)	**NEW**	base
File upload over the TCP socket — fixes the YModem binary stream so a sender's file push works	**FIX**	1 +S2
Probe modelling — real stock / toolsetter / spoilboard surfaces, trips on any axis, honours $6 invert	**NEW**	2
littlefs filesystem + `-format` (persisted to an image) — gives the sim a real on-board filesystem	**NEW**	3 +S1
Settings persistence fix (EEPROM checksum + ATC gate) and $REBOOT via ordered process relaunch	**FIX**	4
3D machine view + live material-removal carve (reads `(TOOL…)` / `(STOCK…)` comments)	**NEW**	5

B	Sim-core base — simulated switches, fast homing, CI	`(in all branches)`	+192/−9 · 4 files

	release		
1	YModem binary stream over the socket	`pr/ymodem-socket`	`+254/-55 · 11 files`
2	Probe modelling (any-axis, stock/toolsetter/spoilboard, $6)	`pr/sim-probe`	`+255/-9 · 4 files`
3	littlefs filesystem + `-format`	`pr/littlefs`	`+489/-17 · 12 files (+ littlefs lib)`
4	Settings persistence fix + `$REBOOT`	`pr/persistence-reboot`	`+535/-26 · 13 files`
5	3D machine view + material-removal carve (superset)	`pr/sim-view`	`+2597/-74 · 26 files`
S1	core: mount littlefs at /littlefs + named-sub search	`core: simulator-littlefs-named-sub`	`submodule`
S2	Plugin_SD_card: YModem init-before-publish fix	`Plugin_SD_card: simulator-ymodem-init-fix`	`submodule`
	Total — base + PRs 1–4 (PR 5 is their integration superset; S1/S2 are submodule-fork commits)		**`+1725/-116 · 44 files`**

Cross-repo dependencies on `stevenrwood/ioSender`. Separate repos — every branch here builds on its own (no compile dependency). The coupling is *runtime / protocol*: a simulator feature is only exercised when the paired ioSender PR drives it over the socket. (ioSender PR numbers refer to that fork's `proposed-prs` tracker.)

This PR	Needs ioSender PR	Why	Strength
PR 1 ymodem-socket	PR 14 sdcard-filesystems · PR 25 atc-macros	The YModem-over-socket fix only matters if a sender uploads files: the file manager (PR 14) and "Install ATC macros" (PR 25) push files via YModem.	**Hard**
PR 2 sim-probe	PR 25 atc-macros	The modelled stock / toolsetter / spoilboard surfaces are probed by the Start-Job / `probe_tfl` tool-length cycle. Manual `G38` works with any sender.	Soft
PR 3 littlefs	PR 14 sdcard-filesystems · PR 25 atc-macros · PR 11 macro-processor	The filesystem is browsed/uploaded (PR 14), written by ATC provisioning (PR 25) and read by `O<name> CALL` forwarding (PR 11).	**Hard**
PR 4 persistence-reboot	PR 15 connect-simulator ("My Machine" only)	Settings persistence backs PR 15's "My Machine" EEPROM mirroring. `$REBOOT` is firmware-internal — any sender can issue it.	Soft
PR 5 sim-view	PR 6 load-folder · PR 15 connect-simulator	The carve reads `(TOOL …)` / `(STOCK …)` comments: PR 6 pushes them on load, PR 15's *always-send-comments* forces them through and is how you connect to the standalone sim. Without them the carve overlay reads "no cutter geometry".	**Hard** (carve)

Base	PR 15 connect-simulator (download only)	The `sim-latest` CI asset feeds PR 15's download path (now vestigial, since that UI was stripped). Homing / switch simulation needs no sender feature.	Soft

Same-repo prerequisites: PR 1 also needs submodule fork S2 (Plugin_SD_card YModem init fix); PR 3 also needs submodule fork S1 (core named-sub `/littlefs` path).

BASE Sim-core: simulated switches, fast homing, CI release

Scope	The first 6 commits (`171a661..741ab27`) shared by **all** proposal branches below.
Files	`src/driver.c, src/driver.h, src/simulator.c, .github/workflows/release-sim.yml`

The stock simulator does not model limit or probe switches, so homing (`$H`) alarms out and probing cannot be exercised at all. This base makes the simulated machine behave enough like real hardware to run those workflows:

- **Simulated limit & probe switches** derived from the tracked machine position, so `$H` and probe moves see real switch transitions.
- **Fast `$H`** — the carriage parks near the home switch instead of seeking the full travel in real time, and `[MSG:Homing]` is emitted at the start.
- **Honor `$5`** (limit invert) in the simulated limit switches.
- **Suppress the hard-limit IRQ during homing** — otherwise a machine with `$21=1` (hard limits enabled) trips `Alarm 8` the instant homing seeks the switch.
- **CI release workflow** (`release-sim.yml`): builds the patched Windows `grblHAL_sim.exe` and publishes it as the rolling `sim-latest` release asset — the build ioSender's "download simulator" path consumes.

PR 1 Fix YModem (binary stream) over the socket so file upload works

Branch	`pr/ymodem-socket`
Depends on	Sim-core base.
Compare	https://github.com/grblHAL/Simulator/compare/master...stevenrwood:Simulator:pr/ymodem-socket

File upload into the simulator (ioSender's *Install ATC macros*, which sends the macro files over YModem) did not work across the simulator's TCP socket.

- **YModem over the socket:** handle the YModem binary stream correctly on the socket transport (the byte path differs from the serial console).
- **Flush per TX burst, not per byte:** the simulator flushed socket output one byte at a time, which stalled the binary transfer; flushing per write-burst lets blocks move.

Pairs with the Plugin_SD_card submodule fork (the init-before-publish race that made block 0 NAK out of sequence). Touches the serial/MCU/platform plumbing: `serial.c`, `simulator.c`, `mcu.c`, `platform_windows.c`/`platform_linux.c`, `main.c`.

PR 2 Probe modelling: any-axis, real surfaces, $6 invert

Branch	`pr/sim-probe`
Depends on	Sim-core base.
Compare	https://github.com/grblHAL/Simulator/compare/master...stevenrwood:Simulator:pr/sim-probe

Extends the simulated probe so ATC tool-length and 3D probing workflows can be rehearsed in the simulator:

- **Trip on any axis** (not just Z), enabling 3D corner / edge probing.
- **Model real surfaces** — stock, toolsetter and spoilboard — so probe moves contact something meaningful at the expected heights.
- **Honor `$6`** (probe invert): an inverted-probe configuration is otherwise read as permanently triggered.

Files: `src/driver.c`, `src/driver.h`, `src/simulator.c`.

PR 3 Add littlefs filesystem support + `-format`

Branch	`pr/littlefs`
Depends on	Sim-core base; the core and Plugin_SD_card submodule forks.
Compare	https://github.com/grblHAL/Simulator/compare/master...stevenrwood:Simulator:pr/littlefs

Gives the simulator a persistent filesystem so ATC macro upload and `O<name> CALL` macro resolution behave like real hardware:

- **littlefs** backed by a host file, wired in via the SD card plugin.
- **Per-block persistence** in the littlefs HAL (rather than rewriting the whole image on every write).
- **Optional case-insensitive name matching** (`LFS_CASE_INSENSITIVE`) so upper-cased O-word labels resolve to lower-case macro files, matching FatFs semantics.
- **Mount at `/littlefs`** and resolve named-sub macros there (the matching core search-path change is the core submodule fork).
- **`-format`** command-line switch to wipe and reformat littlefs on boot.

The line count excludes the vendored `littlefs` library bundled into the build. Files: `src/littlefs_hal.c/.h`, `src/driver.c`, `src/main.c`, `CMakeLists.txt/.tpl`, `.gitmodules` (submodule pointers).

PR 4 Settings persistence fix + `$REBOOT`

Branch	`pr/persistence-reboot`
Depends on	Sim-core base + the littlefs work (PR 3).
Compare	https://github.com/grblHAL/Simulator/compare/master...stevenrwood:Simulator:pr/persistence-reboot

Two lifecycle fixes so the simulator survives restarts and self-reboots like real firmware:

- **Settings persistence:** correct the EEPROM checksum so $ settings survive a restart, and gate the ATC `error_on_no_macro` behaviour.
- **`$REBOOT` (`hal.reboot`):** implemented by relaunching the simulator process in an ordered way, so firmware that reboots itself (e.g. after a settings change) comes back up instead of just exiting.

Files: `src/grbl_eeprom_extensions.c`, `src/driver.c`, `src/main.c`, `src/simulator.c`.

PR 5 3D machine view + material-removal carve (superset = `integration`)

Branch	`pr/sim-view` (= `integration`, `9f7fdab`)
Contains	The sim-core base + PRs 1–4 + the 3D view. This is the branch `sim-latest` is built from.
Compare	https://github.com/grblHAL/Simulator/compare/master...stevenrwood:Simulator:pr/sim-view

The flagship branch: adds an optional OpenGL **3D machine view** to the simulator and a stack of features on top of it. (Most of the bulk is `sim_view.c`.)

- **3D view (`-view`):** a window showing the machine, work envelope and tool; the simulator **runs standalone by default** (3D window on, default port 23, auto-load of the setup/EEPROM), and closing the window shuts the simulator down.
- **Material-removal carve:** a stock heightmap is lowered as the cutter passes (rendered via a GPU display list), shaded by depth and by cutter type (flat / ball / V); with *Reset Stock*, *Flip Stock* (turn over to a fresh top) and a configurable carve resolution. Stock size is taken from a `(STOCK X= Y= Z=)` g-code comment.
- **UI:** a real menu bar (*Settings* / *Format LittleFS* / *Show Log*), an editable Settings dialog (machine description → window title, carve resolution), a dedicated Show-Log window, a carve-status overlay, a build date/time stamp in the title, window-position persistence, and an application icon.
- **ATC / tool table:** `hal.probe.select` (fixes `error:39` on `G65 P5`), end-of-line `(TOOL …)` comment parsing, and a tool-table format spec (`TOOL_TABLE_FORMAT.md`) for the CAM post / add-in.
- **`-setup <file>` fixtures:** define the spoilboard / stock / toolsetter explicitly and write `G28` / `G30` / `G59.3` at boot (decouples the spoilboard from `Z_max_travel`, fixing Alarm 5 on tall-Z machines).

Key files: `src/sim_view.c/.h`, `src/sim_setup.h`, `src/driver.c`, `src/main.c`, `src/sim.rc/sim.ico`, `TOOL_TABLE_FORMAT.md`, `CMakeLists.txt`. Builds with CMake + MinGW (`LITTLEFS_ENABLE=2`).

SUBMODULE core: mount littlefs at /littlefs + named-sub search path

Repo	`grblHAL/core` (fork `stevenrwood/core`)
Branch	`simulator-littlefs-named-sub` @ `fe16cb9` — submodule at `src/grbl`
Compare	https://github.com/grblHAL/core/compare/master...stevenrwood:core:simulator-littlefs-named-sub

The named-sub (`O<name> CALL`) resolver searched only the root filesystem. In the simulator's combined config littlefs mounts at `/littlefs` (root being the SD card), so macros stored on littlefs were never found — the open failure surfaced misleadingly as `error:81`. This fork makes the resolver search `/littlefs`, so uploaded ATC macros resolve. Consumed by PR 3 and PR 5 via the `src/grbl` submodule pointer.

SUBMODULE Plugin_SD_card: YModem init-before-publish race fix

Repo	`grblHAL/Plugin_SD_card` (fork `stevenrwood/Plugin_SD_card`)
Branch	`simulator-ymodem-init-fix` @ `e33441a` — submodule at `src/sdcard`
Compare	https://github.com/grblHAL/Plugin_SD_card/compare/master...stevenrwood:Plugin_SD_card:simulator-ymodem-init-fix

`trap_initial_soh` published `grbl.on_execute_realtime = protocol_loop` *before* initialising `ymodem.next_timeout` / `process`. In the simulator (whose grbl loop runs on a separate thread) that ordering let a spurious timeout NAK fire before block 0, so ioSender retransmitted block 0 out of sequence — the transfer stalled or flooded into an `OutOfMemoryException`. Fixed by initialising the YModem state **before** publishing the realtime hook. This is what makes ATC macro upload work at all; it pairs with PR 1 on the simulator side. Consumed via the `src/sdcard` submodule pointer.

Generated 2026-06-20. Branch tips: `pr/ymodem-socket 0f15ed5` · `pr/sim-probe 4df72ca` · `pr/littlefs 73b3e74` · `pr/persistence-reboot cadca97` · `pr/sim-view 9f7fdab` (= `integration`). Upstream base `grblHAL/Simulator master 171a661`.